



UBA
1821 Universidad
de Buenos Aires



Taller de Sistemas Embebidos (TA134)

FACULTAD DE INGENIERÍA DE LA UNIVERSIDAD DE BUENOS AIRES

TRABAJO FINAL

PRODUCTO MÍNIMO VIABLE:

SISTEMA DE MONITOREO

Autores:

Baldoni, Sofia
Lazo, Sebastian Nahuel
Spadachini, Franco Nicolas
Gomez, Cassandra

Padrón:

106453
106213
110032
109695

Correo:

sbaldoni@fi.uba.ar
slazo@fi.uba.ar
fnspada@gmail.com
Cgomez@fi.uba.ar

Índice

1. Resumen	2
2. Objetivo	2
3. Descripción del Sistema	2
3.1. Descripción del Comportamiento	2
3.2. Componentes involucrados	3
3.3. Arquitectura general	3
3.3.1. Hardware	3
3.3.2. Diagramas Eléctricos	5
3.3.3. Software	6
3.3.4. Diagrama de flujo del algoritmo de actualización de los Sensores	7
3.3.5. Estructura de Cola Circular Utilizada en el Menú	8
3.3.6. Maquina de estado	8
3.4. Tipo de Monitoreo	9
3.4.1. Sensores Incorporados	10
3.4.2. Formato de los Datos	10
3.4.3. Factor de Escalado	10
3.5. Protocolos de Comunicación	10
3.6. Tecnologías utilizadas	11
4. Medición de Parámetros	11
5. Análisis de Consumo	11
6. Factor de Uso de la CPU	12
7. Índice de figuras	13
8. Equipos e instrumental de medición	14
9. Bibliografía	14

1. Resumen

El presente proyecto fue desarrollado en el marco de la asignatura Taller de Sistemas Embebidos de la carrera de Ingeniería Electrónica, aplicando conocimientos teóricos y prácticos adquiridos durante la formación intermedia.

Se trata de un sistema de monitoreo para un vehículo aéreo no tripulado (dron), diseñado para visualizar información relevante como la temperatura, inclinación y altura. Estos datos se obtienen mediante un sensor inercial y se presentan a través de un menú interactivo, que además permite configurar distintos modos de operación y consumo energético.

2. Objetivo

Para garantizar la viabilidad del proyecto se propuso como objetivo cumplir con las siguientes características:

El sistema debe visualizar en tiempo real los parámetros que caracterizan el movimiento del dron cuando se lo solicite mediante el menú interactivo. Se debe poder intercambiar modos de operación desde el menú interactivo.

Garantizar criterios de calidad en el desarrollo del software: eficiencia en el uso de recursos, escalabilidad para futuras mejoras o ampliaciones del sistema, y facilidad de interpretación del código para asegurar su mantenimiento y reutilización.

- Visualización en tiempo real de los parámetros del sistema simulando el comportamiento de un dron.
- Transmisión de datos desde el sistema a una estación base.
- Interfaz básica para monitoreo remoto.
- Implementación de un menú interactivo en pantalla LCD para mostrar los datos.
- Validación de técnicas de programación estructurada y bajo consumo en sistemas embebidos.

3. Descripción del Sistema

3.1. Descripción del Comportamiento

El sistema funciona de manera continua dentro de un lazo principal (super-loop) con una base temporal de 1ms. El microcontrolador lee datos de inclinación a través de un sensor y, en función del ángulo detectado, activa dos LEDs ubicados en la dirección correspondiente.

Además del encendido direccional, el sistema ajusta el brillo de los LEDs proporcionalmente al ángulo de inclinación. Esto se logra mediante modulación por ancho de pulso (PWM) generada por el microcontrolador.

El menú LCD cumple una doble función:

- Permite al usuario habilitar o deshabilitar el funcionamiento de los LEDs.
- Muestra en tiempo real los valores actuales de inclinación en los ejes X e Y.

El sistema está organizado en diferentes modos de operación:

- **NORMAL**: muestra los valores de inclinación y ajusta el brillo de los LEDs en tiempo real.
- **SET_UP**: permite configurar el encendido o apagado de LEDs desde el menú.

3.2. Componentes involucrados

- Placa de desarrollo ST NUCLEO-F103RB
- Microcontrolador STM32F103RBT6
- Placa de prubeas autopercorada
- Pulsadores
- Display LCD 16x2 Segmentos.

3.3. Arquitectura general

Se establecieron las pruebas en base a dos placas experimentales, la primera correspondiente a la estación base del sistema y la segunda correspondiente a una representación de un dron y se desarrollo el código en un entorno de desarrollo con diferentes depuradores incorporados para ayudar a la medición de rendimiento y corrección de posibles errores.

3.3.1. Hardware

La placa base cuenta con el microcontrolador incorporado en una placa de desarrollo con diferentes puertos accesibles para el desarrollo junto con un display lcd para visualizar el menú y los datos obtenidos, por ultimo la placa cuenta con diferentes pulsadores para controlar el sistema en modo usuario.

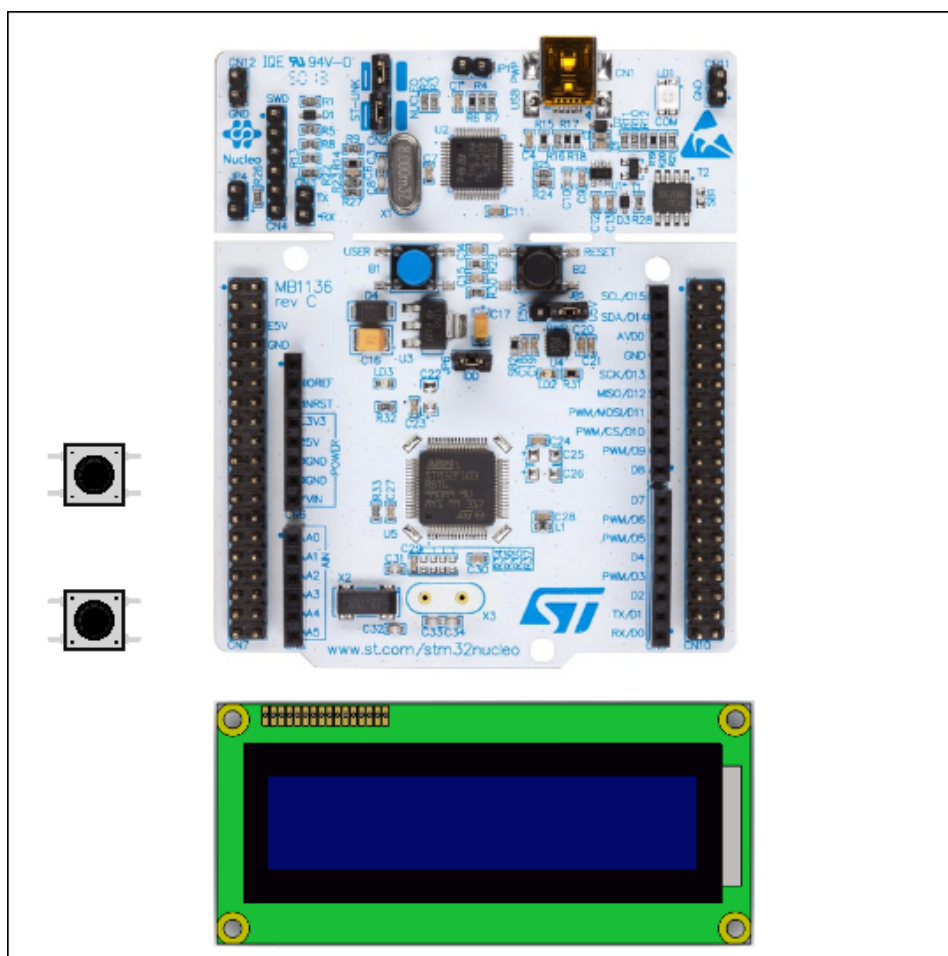


Figura 1: Disposición de la placa de pruebas correspondiente a la base del sistema.

- Placa de desarrollo NUCLEO-F103RB
- Pulsadores
- Display LCD 16x2
- Resistor variable

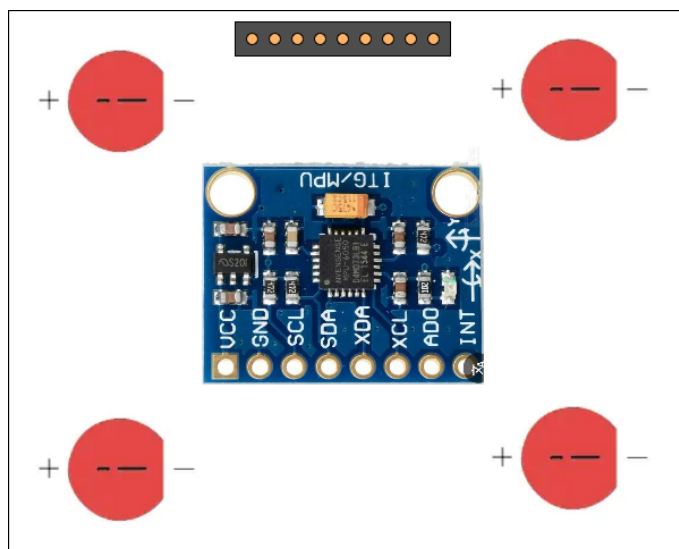


Figura 2: Disposición del prototipo de pruebas del dron.

La segunda placa, correspondiente a la representación del dron, cuenta con cuatro leds indicadores conectados mediante resistores para controlar la corriente suministrada.

- Sensor MPU6050
- Leds indicadores
- Conector socket
- Resistores

3.3.2. Diagramas Eléctricos

A continuación se detallan las principales conexiones eléctricas establecidas entre la placa de desarrollo y los módulos del sistema, basándose en el estándar *Morpho Connector* de la placa NUCLEO-F103RB de STMicroelectronics, empleado para la integración de módulos periféricos y componentes externos tales como sensores, indicadores y elementos de control.

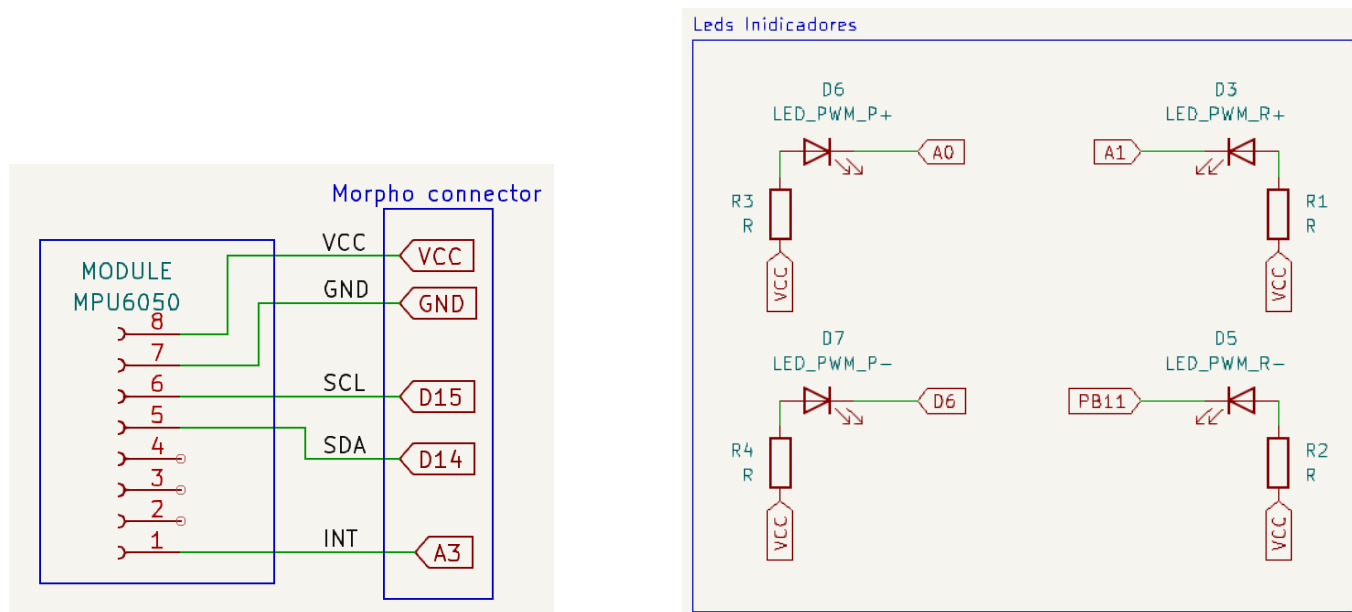


Figura 3: Conexión módulo MPU6050 (izquierda) y conexión LEDs indicadores (derecha).

La placa representativa del dron contiene los LEDs indicadores conectados mediante resistores de $2k2\ \Omega$ y el módulo MPU6050, que agrupa los distintos sensores utilizados.

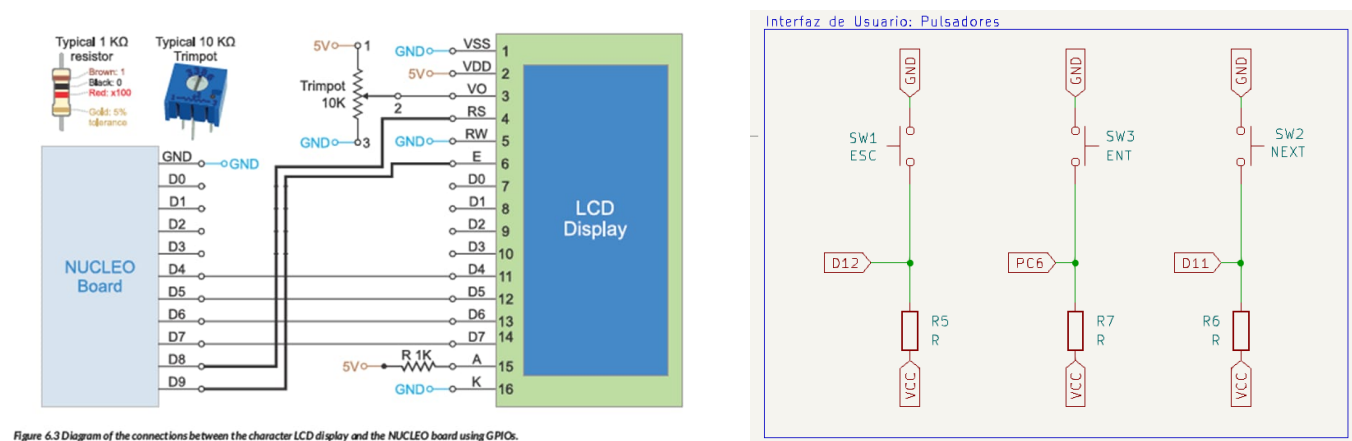


Figura 4: Conexiones del display y de los pulsadores mediante el estandar Morpho connector.

La conexión del display se da con un resistor variable para el ajuste del brillo y resistores de $1k\ \Omega$ para el control de corriente.

3.3.3. Software

El desarrollo del software se realizó utilizando el entorno de desarrollo provisto por STMicroelectronics (STM32CubeIDE). Se adoptó una arquitectura de programación Bare Metal (sin sistema operativo), implementada en lenguaje C, con un enfoque basado en eventos. El sistema fue diseñado a partir de tareas no bloqueantes, estructuradas en módulos que responden a estímulos específicos del entorno.

La lógica del programa sigue el paradigma de escrutar / procesar / actuar, promoviendo una estructura modular y de fácil mantenimiento.

- Lenguaje: C
- Tipo de programación: Bare Metal (sin sistema operativo)
- Modelo de ejecución: Basado en eventos (Event-Triggered System)
- Arquitectura: Modular y estructurada (Escrutar – Procesar – Actuar)

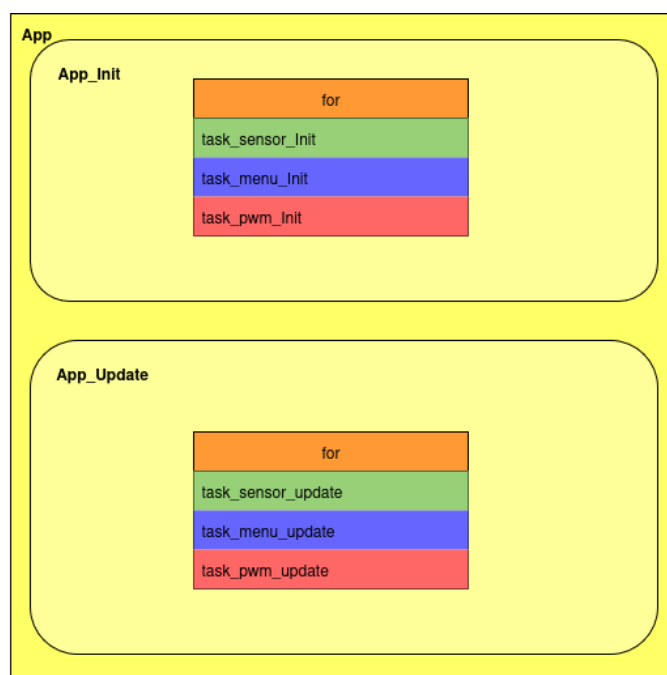
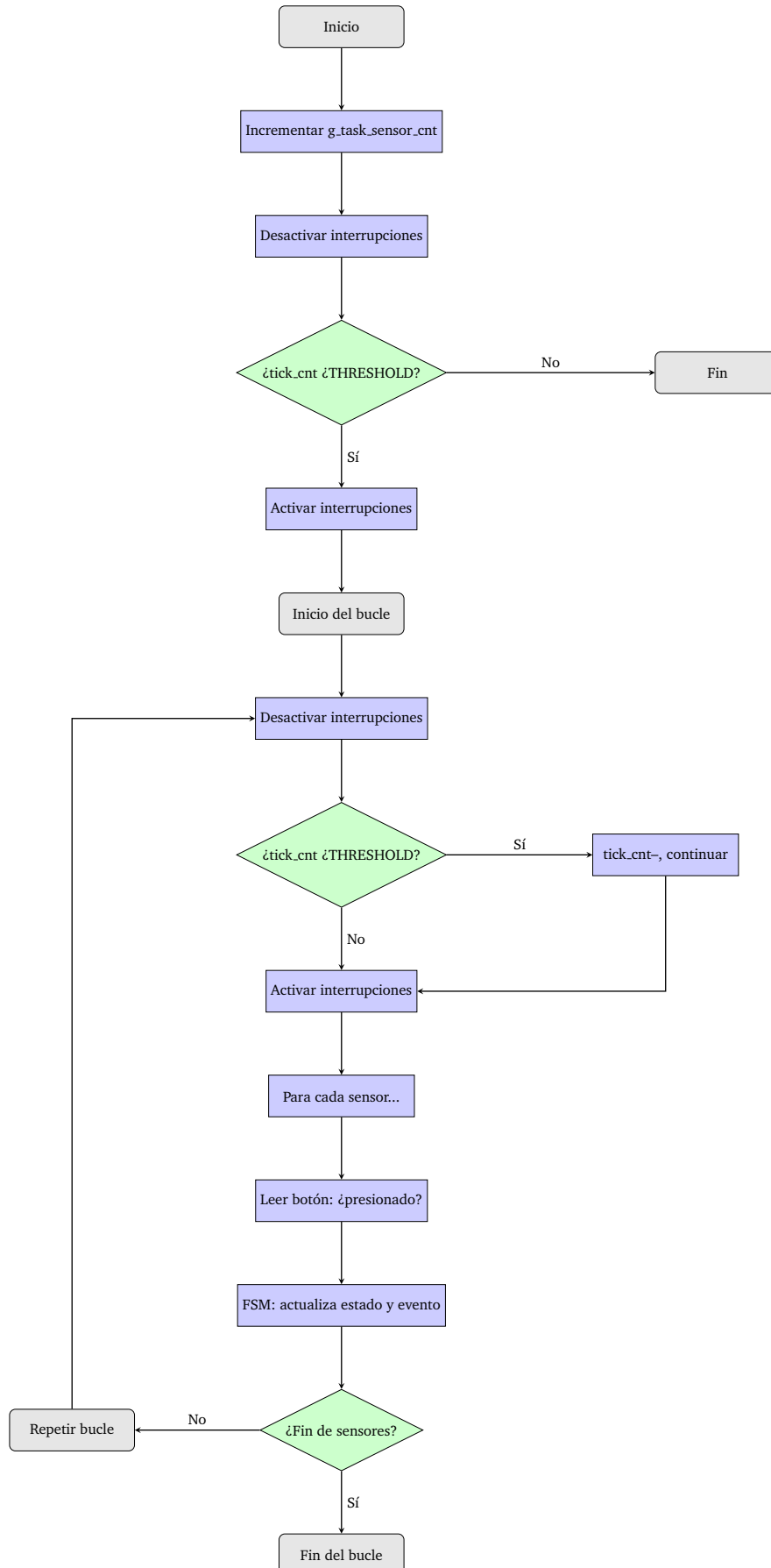


Figura 5: Diagramas de bloques representativo de la aplicación desarrollada.

El sistema está diseñado bajo un esquema orientado a eventos y tareas. En la fase inicial, se crean e inicializan tres tareas principales: Sensores, Menú y PWM.

Posteriormente, en el ciclo principal de ejecución, se actualizan periódicamente estas tareas mediante estructuras iterativas (for), que permiten procesar cada tarea y sus eventos asociados de manera secuencial y organizada. Este enfoque facilita el manejo eficiente y sincronizado de múltiples procesos concurrentes dentro del sistema embebido.

3.3.4. Diagrama de flujo del algoritmo de actualización de los Sensores



3.3.5. Estructura de Cola Circular Utilizada en el Menú

El menú utiliza una **cola circular** para gestionar eventos del tipo `task_menu_ev_t` con una capacidad máxima de `MAX_EVENTS`.

- **Inicialización:** La función `init_queue_event_task_menu()` resetea los índices `head` y `tail` a 0, el contador `count` a 0, y establece todos los elementos de la cola a `EVENT_UNDEFINED`, indicando que está vacía.
- **Inserción de eventos:** La función `put_event_task_menu(event)` añade un evento en la posición `head` de la cola, incrementa `head` (con wrap-around) y actualiza el contador `count`. No verifica si la cola está llena.
- **Extracción de eventos:** La función `get_event_task_menu()` extrae el evento en la posición `tail`, lo elimina (marcándolo como `EVENT_UNDEFINED`), incrementa `tail` (con wrap-around) y decrementa el contador `count`. No verifica si la cola está vacía.
- **Consulta de eventos:** La función `any_event_task_menu()` retorna `true` si existen eventos pendientes (cuando `head` y `tail` son diferentes).

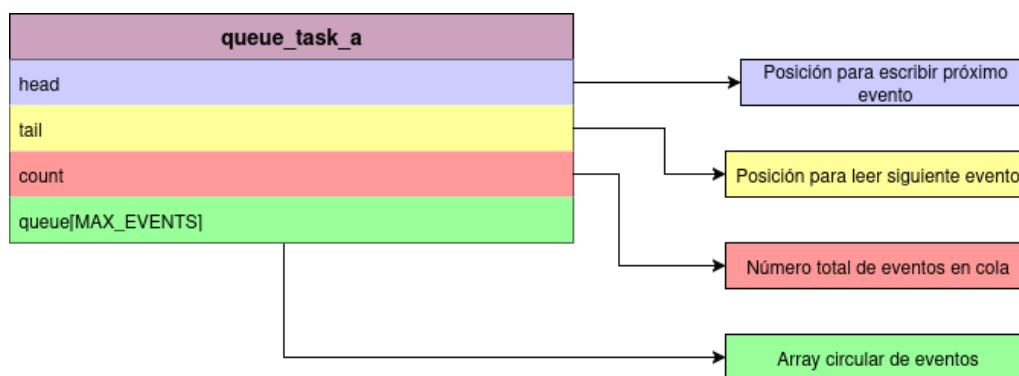


Figura 6: Diagrama de estructura de cola implementada en el menú.

3.3.6. Máquina de estado

Mediante la herramienta `itemis CREATE` se modela el comportamiento del menú mediante la representación de los estados, según el estado actual y una entrada determinada, la máquina realiza transiciones de estado y produce resultados.

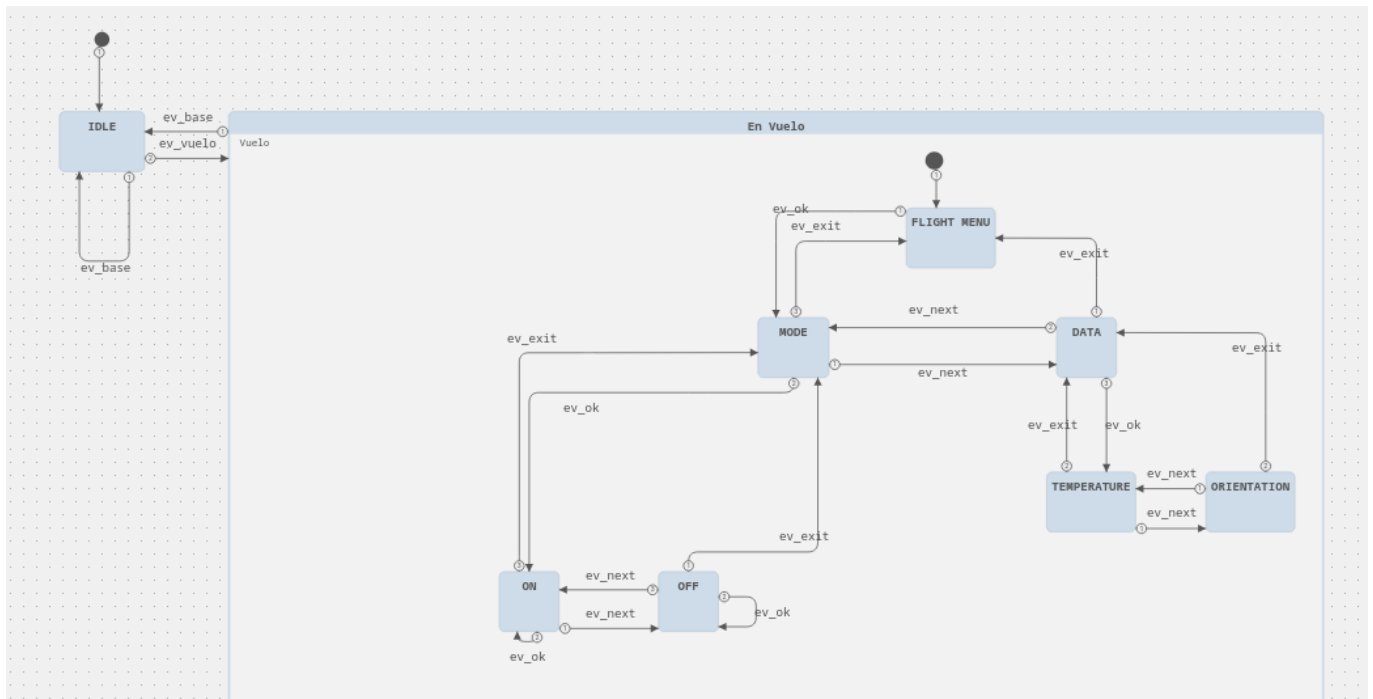


Figura 7: Representación de la maquina de estados del menú interactivo.

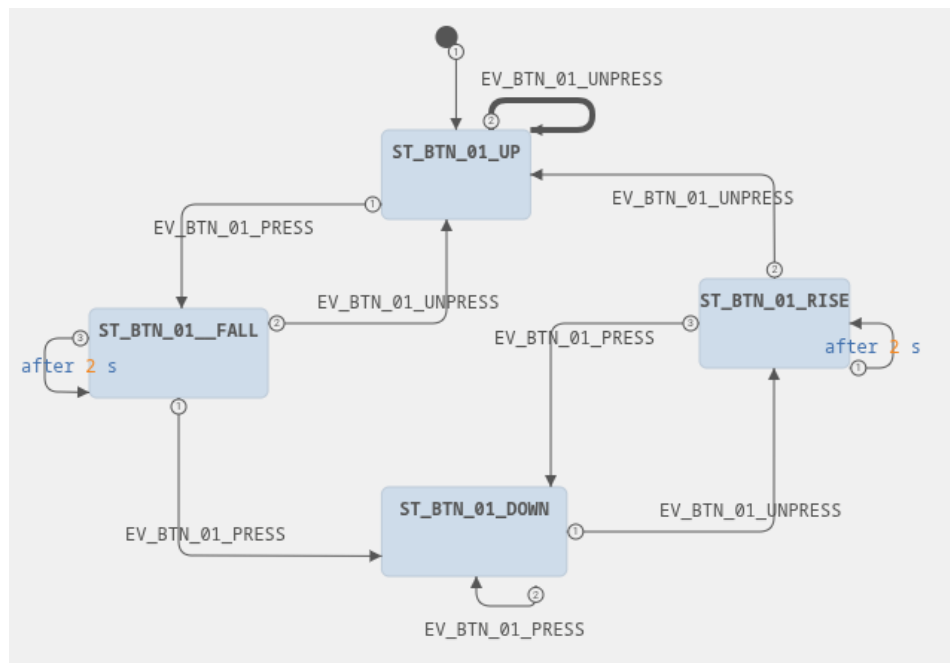


Figura 8: Representación de la maquina de estados de los pulsadores.

3.4. Tipo de Monitoreo

El monitoreo del sistema se basa en el sensor MPU6050, una unidad de medición inercial (IMU) de 6 grados de libertad (6-DOF), que integra un acelerómetro triaxial y un giróscopo triaxial en un solo encapsulado. Este dispositivo permite obtener parámetros dinámicos relevantes del dron, tales como aceleración lineal, velocidad angular, inclinación y orientación espacial. Su integración en sistemas embebidos se realiza típicamente mediante una interfaz I2C.

3.4.1. Sensores Incorporados

- **Acelerómetro (3 ejes):** mide la aceleración lineal en los ejes X, Y y Z. Permite detectar movimientos lineales y estimar la inclinación del dispositivo con respecto al eje vertical.
- **Giroscopio (3 ejes):** mide la velocidad angular (rotacional) en los ejes X, Y y Z. Es fundamental para determinar los cambios de orientación del dron, comúnmente expresados como pitch, roll y yaw.
- **Sensor de temperatura interno (opcional):** aunque no es altamente preciso, puede utilizarse para compensaciones térmicas del sistema.

3.4.2. Formato de los Datos

Los datos generados por el MPU6050 son accesibles a través de registros de 16 bits (2 bytes) cada uno, comenzando en la dirección 0x3B. Una lectura completa abarca 14 bytes e incluye:

- 6 bytes para aceleración (3 ejes)
- 2 bytes para temperatura
- 6 bytes para velocidad angular (3 ejes)

Estos datos son valores crudos (raw) en formato complementado a dos, y deben ser escalados de acuerdo a la configuración de sensibilidad del sensor.

3.4.3. Factor de Escalado

La conversión de los valores crudos a unidades físicas (m/s² o °/s) se realiza mediante un factor de escala que depende del rango de medición configurado:

- **Acelerómetro:** por ejemplo, para un rango de $\pm 2g$, el factor de escala es 16,384 LSB/g.
- **Giroscopio:** para un rango de $\pm 250^\circ/s$, el factor de escala es 131 LSB/(°/s).

Con los datos del acelerómetro, es posible calcular la inclinación del dron utilizando relaciones trigonométricas:

$$\text{Roll} = \arctan\left(\frac{a_y}{a_z}\right) \cdot \frac{180}{\pi}$$
$$\text{Pitch} = \arctan\left(\frac{-a_x}{\sqrt{a_y^2 + a_z^2}}\right) \cdot \frac{180}{\pi}$$

Estos ángulos representan la inclinación del dron en los planos lateral y frontal, siendo claves para el monitoreo, estabilización y control del sistema.

3.5. Protocolos de Comunicación

La comunicación del microcontrolador con los diferentes componentes del sistema se realiza mediante protocolos que establecen la forma en que los datos se envían, como puede ser su codificación y la velocidad con que se envían.

3.6. Tecnologías utilizadas

- IDE: STM32CubeIDE
- Gestor de Versiones: GIT
- Repositorio Remoto: Github
- Depurador: OpenOCD

4. Medición de Parámetros

Tabla 1: Análisis de viabilidad técnica y económica del sistema embebido de monitoreo para dron

Parámetro	Descripción / Método de medición	Valor obtenido / estimado	Unidad
Consumo promedio del sistema	Estimativo	62.26	mA
Precisión de temperatura	Según hoja de datos del sensor y pruebas en laboratorio	± 2	°C
Rango de medición acelerómetro	Hoja de datos del fabricante	± 2	g ($g \approx 9,8 m/s$)
Precisión de altura	Hoja de datos del fabricante	± 10	cm
Costo estimado del hardware	Incluye MCU, sensores, pantalla, conexiones	22.62	USD
Tiempo de desarrollo	Desde planificación hasta prototipo funcional	6	semanas
Escalabilidad del sistema	Posibilidad de integrar nuevos sensores y expandir el menú	Alta	–
Facilidad de mantenimiento	Código modular y documentado, basado en tareas	Alta	–

5. Análisis de Consumo

Para estimar la corriente total del sistema se consideraron los consumos típicos de cada componente:

NUCLEO-F103RB (con ST-LINK) = 40 mA, LCD 16×2 (con retroiluminación) = 16 mA, MPU6050 = 3,9 mA y cuatro LEDs con resistores $2k2 \Omega$ cada uno. Se asume que los pulsadores no consumen corriente en reposo (entradas con pull-up).

La placa se alimenta con 5V mediante la conexión del mini USB, esta placa como máximo puede entregar 120 mA y por cada pin como máximo 25 mA y 3,3 V.

La corriente de cada LED se calcula usando la ley de Ohm, suponiendo $V_{CC} = 3,3$ V y caída en el LED $V_f \approx 2,0$ V:

$$I_{LED} = \frac{V_{CC} - V_f}{R} = \frac{3,3 - 2,0}{2k2} A \approx 590 \mu A.$$

Para los cuatro LEDs:

$$I_{4LEDs} = 4 \cdot I_{LED} \approx 4 \cdot 590 \mu A = 2,36 mA.$$

Sumando los consumos individuales obtenemos el total en el escenario con todos los LEDs encendidos:

$$I_{total} = I_{NUCLEO} + I_{LCD} + I_{MPU} + I_{4LEDs} = 40,00 + 16,00 + 3,90 + 2,36 \text{ mA} \approx 62,26 \text{ mA}.$$

Se reporta $62,2 \text{ mA}$ como consumo estimado en operación típica con LEDs encendidos.

En un escenario de ahorro donde la retroiluminación o los LEDs están apagados, se excluye la corriente de los LEDs (2.36 mA):

$$I_{ahorro} = I_{total} - I_{4LEDs} \approx 62,2 - 2,36 = 59,9 \text{ mA},$$

reportado como 60 mA .

Para referencia rápida, la potencia eléctrica asociada al consumo máximo estimado es:

$$P = V_{CC} \cdot I_{total} = 5 \text{ V} \cdot 62,26 \text{ mA} \approx 311,32 \text{ mW}.$$

Observaciones: los valores son estimativos y dependen de la configuración real (por ejemplo: intensidad de la retroiluminación del LCD, actividad del ST-LINK, variantes del MCU en placa propia). Para una determinación precisa se optaría por medir la corriente con un multímetro en serie, fuente de laboratorio o con un medidor de consumo en la placa final.

Tabla 2: Análisis de Consumo Energético del Sistema de Monitoreo

Modo de Operación	Voltaje (V)	Corriente (mA)	Consumo (mW)
Modo Activo	5	67.5	311.32
Modo No Activo	5	60	300

6. Factor de Uso de la CPU

$$\text{Factor de uso } U = \frac{WCET}{Periodo}$$

Tabla 3: Medición y análisis de tiempos de ejecución (WCET) y cálculo del Factor de Uso (U) de la CPU

Tarea	WCET (ms)	Periodo (ms)	Factor de Uso U
Sensores	6413	93979	0.068
Menú	2619	93979	0.027
PWM	84947	93979	0.9

7. Índice de figuras

1.	Disposición de la placa de pruebas correspondiente a la base del sistema.	3
2.	Disposición del prototipo de pruebas del dron.	4
3.	Conexión módulo MPU6050 (izquierda) y conexión LEDs indicadores (derecha).	5
4.	Conexiones del display y de los pulsadores mediante el estandar Morpho connector.	5
5.	Diagramas de bloques representativo de la aplicación desarrollada.	6
6.	Diagrama de estructura de cola implementada en el menu.	8
7.	Representación de la maquina de estados del menú interactivo.	9
8.	Representación de la maquina de estados de los pulsadores.	9

8. Equipos e instrumental de medición

- Osciloscopio Digital, Gratten, Modelo : GA-1102CAL
- Fuente de alimentación Conmutada, Yihua, Modelo : 305D-IV
- Fuente de alimentación Lineal, LTech, modelo HY-3005E-2
- Multímetro Digital, Proskit, Modelo: MT-1210

9. Bibliografía

Referencias

- [1] René Beuchat, Florian Depraz, Andrea Guerrieri, Sahand Kashani. *Fundamentals of System-on-Chip Design on Arm Cortex-M Microcontrollers*. Arm Education Media, 2021. ISBN: 978-1-911531-334-30 (print), 978-1-911531-35-7 (epub).
- [2] Donald Norris. *Programming with STM32: Getting Started with the Nucleo Board and C/C++*. McGraw Hill TAB, 2018. ISBN: 978-1260031317.
- [3] Brian W. Kernighan, Dennis M. Ritchie. *The C Programming Language*. 2nd ed., Pearson, 1988. ISBN: 978-0131103627.